TUTORIAL 4 – PROGRAMMING KERNEL IN Xv6-RISCV

Objectives:

This tutorial guides you through:

- Compiling and setting up a Unix-like operating system – Xv6
- Creating, compiling and deploying a simple program that runs in kernel-mode

1) Prerequisites & getting the source

What you need (high level)

- A RISC-V newlib toolchain (riscv-gnu-toolchain) on your PATH. The xv6 README points to the upstream riscv-gnu-toolchain (https://github.com/mit-pdos/xv6-riscv)
- QEMU (a riscv64 build). MIT course notes recommend a recent QEMU (e.g. 5.x or newer). pdos.csail.mit.edu
- Standard dev tools: git, make, gcc (host), perl (usys.pl is a perl script).

If you already have a distro package for a riscv cross toolchain / qemu, use that; otherwise you can build the toolchain from riscv-gnu-toolchain (the README in the xv6 repo mentions this requirement). GitHub

Example quick install (Ubuntu-ish) — **adjust to your OS**:

```
# install system packages (Ubuntu/Debian)
sudo apt update
sudo apt install -y git make build-essential perl python3 \
        qemu-system-misc qemu-user \
        # optional: cross toolchain packaged by distro
        gcc-riscv64-unknown-elf || true
```

If you need the upstream GNU toolchain, follow riscv-gnu-toolchain instructions (clone and make newlib) — this builds *lots* of code on your machine. The xv6 README links to the toolchain repo. GitHub

Clone xv6-riscv:

git clone https://github.com/mit-pdos/xv6-riscv.git
cd xv6-riscv

Try to build & run the stock kernel to verify your toolchain + qemu are set up:

```
make clean
make qemu       # builds the kernel + user programs and starts QEMU
```

If that gets you a shell prompt like init: starting sh or a prompt #, you're good to go. (If make qemu errors, the top-level README gives toolchain pointers.)

2) Preparing for a kernel program

What we will do (summary)

1. Add SYS_hello to kernel/syscall.h (pick the next free number).
2. Add an entry("hello"); line in user/usys.pl so the user stub is generated.
3. Add int hello(void); prototype to user/user.h (so user C can call it).
4. Implement the kernel side sys_hello() in kernel/sysproc.c (it will run in kernel and call printf).
5. Add sys_hello to the syscalls[] table in kernel/syscall.c.
6. Replace your user hello program to call hello() (the syscall), rebuild and run.
7. Edit top-level Makefile, adding the program entry to UPROGS tag

This approach runs your code inside the kernel when the syscall executes.

Before you start — find the next free syscall number

Open kernel/syscall.h and note the highest #define SYS_xxx value. The new SYS_hello must use the next integer.

Quick command to show the last SYS_ number (run from xv6 root):

```
grep -n "define SYS_" kernel/syscall.h | sed -n '$p'
```

This prints the last #define SYS_... line. If it shows e.g.

#define SYS_write 22

then the next free number is 23. Use that number in the change below.

(If your syscall.h already contains a SYS_hello — skip the add and use the existing number.)

Add it to the Makefile

Open the top-level Makefile and find the list that begins with:

```
UPROGS=\
	_cat\
	_echo\
	_forktest\
```

Add your new program:

_hello\

(Make sure it's listed with a backslash at the end like the others.)

3) Concrete edits

Below are the exact file changes. Put each snippet into the named file, replacing or appending as noted.

kernel/syscall.h — add the define

Open kernel/syscall.h and append **one line** near the other SYS_ defines. Use *the next free number* you found above.

Example (if next number is 23):

#define SYS_hello 23

(If your last SYS_ was N, use N+1 instead of 23.)

user/usys.pl — add the stub entry

Open user/usys.pl and add entry("hello"); alongside the other entry("..."); lines.

Example context:

```
entry("fork");
entry("exit");
entry("wait");
...
entry("sleep");
entry("hello");    # <-- add this line
```

Then save it.

user/user.h — add user prototype

Open user/user.h and add the prototype (anywhere with other prototypes):

```
int hello(void);
```

kernel/sysproc.c — implement sys_hello

Edit kernel/sysproc.c. Add sys_hello near the other sys_* functions (for example next to sys_getpid, sys_fork, etc.). Use the same include style as other functions.

Add this snippet (paste into the file in a logical spot — e.g., after other sys_ functions):

```
#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "proc.h"

// Kernel-side syscall implementation
uint64
sys_hello(void)
{
  struct proc *p = myproc();
  // A kernel-side message — printed with cprintf (runs in kernel)
  printf("kernel: hello() called by pid %d (running in kernel)\n", p->pid);

  // Optionally perform other kernel-only actions here, e.g.:
  // - allocate kernel memory
  // - inspect process fields
  // Keep it short and safe.

  return 0;  // return value visible to the user caller
}
```

Notes:

- printf prints from kernel (visible in QEMU console).
- myproc() returns the current proc pointer (ensure no duplicate struct proc definitions exist; if earlier edits caused duplicates, remove them so proc.h is the single definition).

kernel/syscall.c — wire the syscall into the table

Open kernel/syscall.c. At the top, add an extern if needed (some xv6 variants don't require it, but it's safe):

```
extern uint64 sys_hello(void);
```

Then in the syscalls[] array mapping (look for a list like [SYS_fork] sys_fork, …), add:

```
[SYS_hello]   sys_hello,
```

Put it at index SYS_hello (the array is keyed by syscall number macro, so ordering in the source is flexible as long as you use the macro).

Write a program to call syscall

Edit your user hello program (file user/hello.c) with this code. This program calls the syscall wrapper hello() which user/usys.pl will generate into user/usys.S:

```
// user/hello.c
// Call kernel hello syscall.

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user.h"

int
main(void)
{
  // call kernel-side hello
  hello();

  // We can also print from user side if desired
  printf("user: hello() returned, back in user mode\n");
  exit(0);
}
```

4) Rebuild the project

Run these commands from the xv6 top-level directory:

```
# regenerate syscall stubs, rebuild, and run
make clean
make user/usys.S    # explicitly regenerate user syscall stubs
make
make qemu
```

make user/usys.S runs user/usys.pl and generates user/usys.S. You should see that usys.S contains a stub for hello.

Start xv6 (if not already started with make qemu) and run:

```
$ hello
```

You should see in the QEMU console the kernel message printed by cprintf, something like:

```
kernel: hello() called by pid 2 (running in kernel)
user: hello() returned, back in user mode
```

kernel: lines come from cprintf (kernel). The user: line is printed after the syscall returns to userland.

5) Troubleshooting checklist

If you see build or runtime errors, check the following:

1. **Duplicate struct proc** — If you previously edited kernel headers and accidentally duplicated struct proc, remove the duplicate so kernel/proc.h is the single definition. Use:

```
grep -nR -- "struct proc {" kernel || true
```

There should be only one hit (kernel/proc.h). If there are more, delete the accidental copy.

2. **Missing user/usys.S or missing sleep earlier problems** — Ensure user/usys.pl contains entry("hello"); and re-run make user/usys.S. Confirm the generated file contains hello:

   ```
   grep -n "hello" user/usys.S || true
   ```

3. **Syscall number conflict** — Make sure SYS_hello is assigned the next unused number. If you accidentally used an existing number, the wrong syscall will be invoked.
4. **Compile errors about cprintf or myproc** — Ensure kernel/sysproc.c includes defs.h and proc.h as in the snippet above.
5. **Linker undefined reference to hello when linking user binary** — That means user/usys.S did not generate the stub. Re-run make user/usys.S and verify.
6. **Kernel messages not appearing** — In QEMU, kernel cprintf output appears in the QEMU console window. If you launched QEMU in a way that doesn't show the console, run make qemu in a terminal.

6) Your exercise

Write another program to:

1. Allocate memory in kernel
2. List process table entries